

output shape of Expectation layer

<https://github.com/tensorflow/quantum/issues/284>

ROW 61

output shape of Expectation layer #284

New issue

Closed

#388



refraction-ray opened on Jul 1, 2020

Expectation layers can feed with batch of circuits, symbol values and measurement operators. However, the output shape shows some conflicts with different combination of batches in the input and even segfault may occur. See the following demo:

```
from sympy import symbols
a, b = symbols("a b")
c1 = circq.Circuit()
c1.append(circq.rx(a)(circq.GridQubit(0,0)))
c1.append(circq.rx(b)(circq.GridQubit(1,0)))
c2 = circq.Circuit()
c2.append(circq.ry(a)(circq.GridQubit(0,0)))
c2.append(circq.ry(b)(circq.GridQubit(1,0)))
m1 = circq.Z(circq.GridQubit(0,0))*circq.Z(circq.GridQubit(1,0))
m2 = circq.Y(circq.GridQubit(0,0))*circq.Y(circq.GridQubit(1,0))

expectation = tfq.layers.Expectation()

print(expectation(c1, symbol_names=[a, b], operators=[m1, m2])) # shape 1,2 # ?,op dimension
print(expectation(c1, symbol_names=[a, b], symbol_values=[[0.,0.]], operators=[m1, m2])) # shape 1,2 # ?, op
print(expectation(c1, symbol_names=[a, b], symbol_values=[[0.,0.],[1.,1.]], operators=[m1, m2])) # shape 2,1 # value
print(expectation(c1, symbol_names=[a, b], symbol_values=[[0.,0.],[1.,1.]], operators=[m1, m2])) # shape 2,2 #
print(expectation([c1, c2], symbol_names=[a, b], operators=[m1, m2])) # 2,1 # circuit dimension, op dimension
print(expectation([c1, c2], symbol_names=[a, b], operators=[m1, m2])) # 2,2 # circuit dimension, op dimension
print(expectation([c1, c2], symbol_names=[a, b], symbol_values=[[0.,0.], [1.,1.]], operators=[m1, m2])) # shap
print(expectation([c1, c2], symbol_names=[a, b], symbol_values=[[0.,0.]], operators=[m1, m2])) # segmentation
```

The batch of symbol values and batch of circuits both take the first dimension of outputs. And when both are in batch form, circuits beyond the first one seem to be directly omitted by expectation layers. Besides, I met segmentation fault for the last line: when three things all in batches and only one set of symbol values are provided.

Any thoughts on this?

MichaelBroughton on Jul 2, 2020 · edited by MichaelBroughton

Edits Collaborator

Thanks for raising this issue. Couple thoughts:

1. Creating one layer and calling it over and over with different input signatures is not a natural workflow for TF and Keras, usually one creates a single layer that in turn gets linked into a larger computation graph. From there input and output pairs are sent through the graph.
2. The reason you are getting this strange behavior is because each time you call the layer after it's creation you are forcing it to save some small amount of internal state (under the assumption that it will not be called with a different signature) in order to function correctly in a compute graph. For example in the first print statement the layer is assuming that because you didn't provide any `symbol_values` that you want the layer to manage that for you. From there it will create it's own learnable variable for `symbol_values` to match the size of `symbol_names`. In the third print statement, what is happening behind the scenes is that the layer is creating a tile operation to generate many copies of `c1` with batch size equal to the batch size of `symbol_values`. Eventually what happens is that the incompatibility of all these different states eventually puts the layer into a completely broken state.

So to fix your issues it's likely that you just need to do something like this:

```
print(tfq.layers.Expectation()(stuff))
print(tfq.layers.Expectation()(other_stuff))
```

The reason we opted for this behavior is because it gives the user a bit more flexibility in how they call into the layer at the cost of knowing that things might break if they used it this way. The alternative to this would be something more rigid like this:

```
tfq.layers.Expectation(optional_circuit, optional_operators)(fixed_signature_dictated_by_optional_params)
```

Assignees

MichaelBroughton
balopat

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Fix certain invalid inputs causing segfault on expectation ops.
tensorflow/quantum

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



This would force the user to have to remember, "OK, which optional constructor parameters give me the call signature that I want" as apposed to "OK, I'm gonna send my stuff in to the call and it should just work".

3. The segfault is pretty alarming. Even if the library breaks we should never be segfaulting. @zaqqwerty do you want to track down why this happening ? Maybe we can create some kind of warning for the user that comes up if they try to use multiple different call signatures on the same layer object ?

Does this help clear things up ?



 MichaelBroughton self-assigned this on Jul 2, 2020



refraction-ray on Jul 2, 2020

Author ...

@MichaelBroughton , thanks for your reply.

I changed `expectation` in the above code with `tfq.layers.Expectation()` and the shapes of all the outputs as well as the last segfault persist. So the issues 1. circuit batch and symbol values batch both take the same dimension in the outputs and hence may be conflict and 2. segfault in the last line persist.



MichaelBroughton on Jul 2, 2020

Collaborator ...

Ok In that case we should definitely look into the segfault issue.

┆ circuit batch and symbol values batch both take the same dimension in the outputs and hence may be conflict

What exactly do you mean by this ? I ran the snippet above just now and saw that there were different output shapes, but that is to expected since the input signatures are different.



refraction-ray on Jul 2, 2020 · edited by refraction-ray

Edits Author ...

You can check the comments after each print line in the code. Basically, there are three things can be provided in batch (list) for expectation layers. They are input(circuits), operator(measurement), and symbol_values. However, the output shape rank is always 2. The second dimension is determined by operator batch, while the first dimension is determined either by input batch or symbol value batch. When you provided the three arguments all in list form, the output shape is still rank 2, and there will be some conflicts.

Specifically, for this line

```
print(expectation([c1, c2], symbol_names=[a, b], symbol_values=[[0.,0.], [1.,1.]], operators=[m1, m2]))
```

I expect the output should be in 2(2 circuits) times 2(2 symbol values) times 2(2 measurements) shape, while the output is 2 by 2



MichaelBroughton on Jul 2, 2020

Collaborator ...

This is the desired behavior. In the case of the circuits input **not being a list or tensor**, the layer will create multiple copies of the circuit in order to resolve each variable value into it in a 1:1 fashion:

```
print(tfq.layers.Expectation()(c1, symbol_names=[a, b], symbol_values=[[0,0], [1,1]], operators=[m1, m2])
```

This gives output shape 2,2 where `out[i][j] = [[<c1([0,0]) | m1 | c1([0,0])> , <c1([0,0]) | m2 | c1([0,0])>],
[<c1([1,1]) | m1 | c1([1,1])> , <c1([1,1]) | m2 | c1([1,1])>]]`

Whereas if I do something like this where I pass a `list/tensor` of circuits we assume that you will give a good 1:1 mapping with `symbol_values` :

```
print(tfq.layers.Expectation()([c1, c2, c3], symbol_names=[a, b], symbol_values=[[0,0], [1,1], [2,2]], operators=[m1, m2]))
```

The output shape will be 3,2 where: `out[i][j] = [[<c1([0,0]) | m1 | c1([0,0])> , <c1([0,0]) | m2 | c1([0,0])>],
[<c2([1,1]) | m1 | c2([1,1])> , <c2([1,1]) | m2 | c2([1,1])>], [<c3([2,2]) | m1 | c3([2,2])> , <c3([2,2]) | m2 | c3([2,2])>]]`

To use your example when you run:

```
# This should be [2,2]
print(tfq.layers.Expectation()([c1, c2], symbol_names=[a, b], symbol_values=[[0,0], [1,1]], operators=[m1, m2]))

# This should be [1,2] and look like the first row
print(tfq.layers.Expectation()(c1, symbol_names=[a, b], symbol_values=[[0,0]], operators=[m1, m2]))
# This should be [1,2] and look like the second row
print(tfq.layers.Expectation()(c2, symbol_names=[a, b], symbol_values=[[1,1]], operators=[m1, m2]))
```

Does this make sense ?



refraction-ray on Jul 2, 2020

Author ...

Wow, this makes tons of sense. Thanks for your detailed clarification! And I believe this is why there is segfault when circuit batch mismatches with symbol values batch (although as you said, segfault should never happen in robust implementations)



aburacze mentioned this on Jul 12, 2020

[Memory allocation errors \(Python 3.7 + Windows and Linux\) #296](#)

MichaelBroughton assigned **balopat** on Jul 14, 2020

MichaelBroughton mentioned this on Sep 17, 2020

[Fix certain invalid inputs causing segfault on expectation ops. #388](#)

MichaelBroughton closed this as **completed** in **#388** on Sep 18, 2020

jaeyoo added a commit that references this issue on Mar 30, 2023

Merge pull request [tensorflow#284](#) from quantumlib/initial-states-qtra. ...

Verified 92ceee4

Fix certain invalid inputs causing segfault on expectation ops. #388

<> Code

Merged MichaelBroughton merged 4 commits into master from segfault_fix on Sep 18, 2020

Conversation 3 Commits 4 Checks 0 Files changed 10 +87 -14

MichaelBroughton commented on Sep 17, 2020 • edited Collaborator

Fixes #284.

- Fix certain invalid inputs causing segfault on expectation ops. 2383052
- MichaelBroughton marked this pull request as draft 5 years ago
- MichaelBroughton added 2 commits 5 years ago
 - format. 8c699a2
 - C++ format. 2840743
- MichaelBroughton requested a review from zaqqwerty 5 years ago
- MichaelBroughton marked this pull request as ready for review 5 years ago

zaqqwerty suggested changes on Sep 17, 2020 View reviewed changes

zaqqwerty left a comment Contributor

In the original issue, the last line in the OP's example code is:

```
print(expectation([c1, c2], symbol_names=[a, b], symbol_values=[[0., 0.1], operators=[m1, m2]]) #  
segmentation fault
```


Can this scenario be tested in the expectation and sampled expectation layer test files?

- ghost previously approved these changes on Sep 18, 2020 View reviewed changes
 - A feedback. b4d2ce4
- MichaelBroughton dismissed ghost's stale review via b4d2ce4 5 years ago

MichaelBroughton commented on Sep 18, 2020 Collaborator Author

Good idea @zaqqwerty, added tests for all layers. Our keras layer tests are kind of messy, maybe we should open an issue to clean things up a bit.

Reviewers

zaqqwerty ✓

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

output shape of Expectation layer

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

2 participants

